CHAPTER 4 - Session Management

4.1 Objectives

This topic is aimed to explain about session management in servlets using different techniques and with example programs. The objectives of the course is to enable the trainees understand the below topics

- What is a Session?
- Session Management using various techniques
- Implementing session tracking
- Understanding the session-tracking API

4.2 Audience

This document is designed for Java EE Learners and programmers with a need to understand the concept of Session Management in Web Applications. The trainees who have understood the basics of Java technology, programmed Core Java programs, and implemented Servlets are the intended audience.

4.3 Prerequisites

Associates should have gone through all the concepts of the core skills phase, specifically the core java concepts and fundamentals of Servlets.

4.4 Course Content

4.4.1 Introduction

HTTP- A stateless protocol, when a client sends a request, the server sends back a response but does not keep any information about that request and the client state.

In most web applications a client has to access various pages before completing a specific task and the client state should be kept along all those pages.

Example 1: A customer wants to buy some items in an on-line store. She/he should add each one of them to his/her shopping cart and, at the end, pay for them in order to complete the purchase

Example 2: A customer wants to make a reservation for a specific event: he/she should choose the event, the city, the theater in which it takes place, the schedule, the seat and, finally buy one or more tickets for that specific event.

Both examples involve a conversation between the customer and the web application (at the server side).The server must maintain, at every moment, the client's conversational state (i.e., the information that the client has sent to the server along his/her interaction with the web application).

How to remember the client's conversational state using a stateless protocol?

In a web application we may maintain the client state by means of different techniques like

- Hidden fields
- URL rewriting
- Cookies
- Session Management API

4.4.2 Importance of Session Management

Session management is to use the API provided by your application server to identify the user across multiple page requests.

Note that every server has its own set of API for session management, since we are talking about 'Managing Sessions with Java Servlets', we will be making use of the Servlet API which almost all of the Java application servers support

Importance of Session management

It is important to manage each user's activity and information separately when there are thousands of concurrent users using the web application. We implement session to identify each user and perform user based operation.

Following are only **some** of the usage:

Security:-

You can allow membership based access to your site thus making sure that only members get to see special content on your site. After logging in you can identify members from non-members by setting an attribute on the user session to some value which avoids log in again and again.

Customizations:-

You can allow site visitors to customize the look and feel of your site, thus show each user a different view of your site. You can also show different content to different users depending on their preferences.

User Behavior:-

You can log user behavior like how many ad views have been shown to the user. If incase many have been shown with no response from the user then it is time to change that ad.

4.4.3 Types of Session Management – with examples

We will be seeing the various techniques of session management with an example of retrieving the number of page hits

4.4.3.1 Hidden fields

The client state is passed from the server to the client and back to the server in a hidden field of a form.

Hidden Form Field is:

- Simplest technique to maintain the state of an end user.
- Embedded in an HTML form.
- Not visible when you view an HTML files in a browser window.
- Not able to maintain the state of an end user when it encounters a static document.

For Example:

```
<%
        int n;
        if (request.getParameter("counter")==null) n=1;
        else{
        n=Integer.parseInt(request.getParameter("counter"))+1;
```

```
        }
%>
<html>
<h1>Page counter using hidden fields</h1>
<form action="index.jsp" method="get" >
This is your access number <%=n%> <p>
<input type="hidden" name="counter"
value=<%=String.valueOf(n)%>>
<input type="submit" value="New access">
</form>
</html>
```

4.4.3.2 URL Rewriting

The client state is passed from the server to the client and back to the server in the query string, accompanying the URL.

URL Rewriting

- Maintains the state of end user by modifying the URL.
- Is used when the information to be transferred is not critical.
- Cannot be used for maintaining the state of an end user when a static document is encountered.

- User can't modify the URL.

Example:

```
<%
        int n;
        if (request.getParameter("counter")==null)
                n=1;
        else
        {n=Integer.parseInt(request.getParameter("counter"))+1;
        }
%>
        <html>
        <h1>Page counter using URL rewriting</h1>
        This is your access number <%=n%>
        <p>
        <a href="index.jsp?counter=<%=String.valueOf(n)%>">
        New access</a>
        </html>
```

4.4.3.3 Cookies

A cookie is a pair (key-string, value-string) which is:

- Sent by the server to the client as a part of the response to a client request
- Stored by the client browser (when it receives the cookie from the server)
- Sent back by the client as a part of subsequent requests to the same server
- Cookie consists of a single name-value pair. This pair may be followed by attribute-value pairs that are separated by semicolons. Used by the server to find out the computer name, IP address, or any other details of the client computer.
- A small amount of information sent by a servlet to a Web browser and is saved by the browser. It can be later sent back to the server also.
- Have a name, a single value, and optional attributes such as comment, path and domain qualifiers, a maximum age, and a version number.

Cookies can be used to store the state reached by the client in the previous request so that it can be restored in the next one.

Example:

The cookie (accessNb,”12”) may mean that as a consequence of the last request (access to a page p) the number of accesses to p has been 12.

When the client sends another request to access p the server will increase that number by one and will send the new cookie (accessNb,”13”) to the client.

Cookie class operations

• **Cookie(java.lang.String name, java.lang.String value)**

Constructs a cookie with a specified name and value.

• **java.lang.String getName()**

Returns the name of the cookie.

• **java.lang.String getName()**

Returns the name of the cookie.

• **java.lang.String getPath()**

Returns the path on the server to which the browser returns this cookie.

• **java.lang.String getValue()**

Returns the value of the cookie.

• **void setMaxAge(int expiry)**

Sets the maximum age of the cookie in seconds.

• **void setPath(java.lang.String uri)**

Specifies a path for the cookie to which the client should return the cookie.

• **void setValue(java.lang.String newValue)**

Assigns a new value to a cookie after the cookie is created.

Cookies support in other classes

Interface HttpServletRequest

• **Cookie[] getCookies()**

Returns an array containing all of the Cookie objects the client sent with this request.

Interface HttpServletResponse

• **void addCookie(Cookie cookie)**

Adds the specified cookie to the response.

Example

Cookies-An access counter

```
<%
        final int YEARSECS=60*60*24*365;
        int n;
        Cookie[] cookies=request.getCookies();
        String accessnb=null;
        if (cookies!=null)
        {
                for (int i=0; i<cookies.length;i++)
                {
                        Cookie c=cookies[i];
                        if ((c.getName()).equals("accessNb"))
```

```
                        accessnb=c.getValue();
                }
        }
        if (accessnb==null)
        {
                n=1;
        }
        else
        {
                n=Integer.parseInt(accessnb);
        }
        Cookie c=new Cookie("accessNb",String.valueOf(n+1));
        c.setMaxAge(YEARSECS);
        response.addCookie(c);
%>
        <html>
        <h2> Access counter </h2>
        <p>
        This is a version without session support.A cookie is used to store (in the
        client side) the value of the last access number.
        <p>
        <p>
        This is your access number <%=n%>
        <p>
```

```
Click <a href="index.jsp"> here </a>
</html>
```

4.4.3.4 HttpSession

Now, we know how to track user sessions pro-grammatically, using URL rewriting, Cookies, and hidden form fields respectively.

Each of these techniques required some unique string to be exchanged between the client and the server, so that the server could recognize the client.

However, there are certain complications with all three approaches as mentioned below:

- **Hidden Field**: This method can't be used with links because it needs the form to be submitted every time request is made from the client to the server with the hidden field. Also it's not secure because we can get the hidden field value from the HTML source and use it to hack session. We can only use textual information with hidden fields.
- **URL Rewriting :** This is very tedious because we need to keep track of this parameter in every response and make sure it's not clashing with other parameters
- **Cookies:** We can maintain a session with cookies but if the client disables the cookies, then it won't work.

The Java Servlet API has provisions to look at the session handling the Servlet API provides the following facilities for managing sessions:

- ✓ Management of session lifecycle, including session creation and termination.
- ✓ Management of session state.

The Java Servlet API provides the following interfaces and classes for session handling in the **javax.servlet.http package:**

- **javax.servlet.http.HttpSession** :- Provides an abstraction of a session
- **javax.servlet.http.HttpSessionListener:**- Handles events associated with session creation and termination (lifecycle events).

- **javax.servlet.http.HttpSessionBindingListener** :- Handles events associated with binding and unbinding state for sessions.

HttpSession interface allows servlets to

- ✓ **View and manipulate information about a session**, such as the session identifier, creation time, and last accessed time
- ✓ **Bind objects to sessions**, allowing user information to persist across multiple user connections

Sessions are represented by an HttpSession object. You access a session by calling the

HttpServletRequest.getSession() or HttpServletRequest.getSession(boolean) method of a request object.

This method returns the current session associated with this request, or, if the request does not have a session, it creates one (unless boolean argument is false).

Session creation/identification:

The method service (request, response)/doGet(request,response)/doPost(request, response) should incorporate the following sentence:

HttpSession session=request.getSession();

Creates a session (with a new identifier) if request does not contain any session identifier.

Otherwise, it retrieves the session identifier attached to request (by means of a cookie or a parameter in a query string) and uses it to build an object of class HttpSession.

• HttpSession session=request.getSession(create); (create is a boolean variable here)

If create=true, the same as before

If create=false, retrieves an already existing session but it does not create a new one if request does not have any session identifier attached to.

Session destruction

Sessions are automatically cancelled (and their resources are deallocated) when they reach a certain timeout.

- This timeout is specified in the configuration file **web.xml** of the server container. In the case of **Tomcat:**

```
<web-app>
...
<session-config>
<session-timeout> 30 </session-timeout>
</session-config>
...
</web-app>
```

In this case, the session timeout is established in 30 minutes.

- It is possible to change the timeout for a specific session using the method

setMaxInactiveInterval(n) of the interface HttpSession:

```
session.setMaxInactiveInterval(300);
```

This states the timeout for this session to 300 seconds(5 minutes).

Sessions can be explicitly cancelled by using the method invalidate():

```
session.invalidate()
```

Example

```
public class Counter extends HttpServlet {

public void doGet(HttpServletRequest request,HttpServletResponse response) throws IOException, ServletException {

doPost( request, response ) ;

}

public void doPost (HttpServletRequest request,HttpServletResponse response) {

        try {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        HttpSession session=request.getSession();

        //getSession is responsible for including the cookie JSESSIONID to the response

        if (session.getAttribute("counter")==null)

        session.setAttribute("counter", new Integer(1));

        int c= ((Integer)session.getAttribute("counter")).intValue();

        out.write("\n\n");

        out.write("<html>\n\n");

        out.write("<h2> Access counter");
```

```
        out.write("</h2>\n\n");

        out.write(" \n\nThis is your access number ");

        out.print(c);

        out.write("\n\n");

        session.setAttribute("counter",new Integer(c+1));

        out.write("\n");

        out.write("<p>\nClick \n\n\n");

        out.write(

        "<a href=\"" +

        "counter\">"+

        "here ");

        out.write("</a>\n\n");

        out.write("</html>\n");

        if (c>=5) session.invalidate();

} catch (Exception ex)

         {

        ex.printStackTrace();

        }

}
```

```
}
```

Note: Session management can be applied in login and logout functionality. For details follow the link http://www.javatpoint.com/servlet-http-session-login-and-logout-example

4.4.4 Summary

In this topic you have learnt about

- Session and different session management techniques
- Using the various Servlet SessionAPI